

Preparation aux Entretiens

Algorithmes & Structures de donnees

Comment aborder un probleme de code,
les 11 patterns essentiels, et les pieges a eviter.

Table des matières

1	Methode : comment coder en entretien	2
2	Hash Map / Dictionnaire	3
3	Hash Set	3
4	Two Pointers (deux pointeurs)	3
5	Sliding Window (fenetre glissante)	4
6	Prefix Sum (somme prefixee)	4
7	Sorting (tri)	5
8	Binary Search (recherche dichotomique)	5
9	Stack (pile LIFO)	6
10	Monotonic Stack (pile monotone)	6
11	Backtracking (retour sur trace)	7
12	Linked List (liste chaine)	7
13	Fiche recapitulative des complexites	9

1. Methode : comment coder en entretien

Ne jamais foncer sur le clavier. Un entretien evalue autant ton *raisonnement* que ta solution. Suis systematiquement ces six etapes :

1. **Comprendre** – reformule l'enonce ; clarifie les contraintes (taille, valeurs, doublons, donnees trieées?).
2. **Exemples** – ecris 1 ou 2 exemples a la main, y compris les cas limites (tableau vide, un seul element).
3. **Force brute** – enonce la solution naive et sa complexite. Tu as toujours une base sur laquelle t'appuyer.
4. **Optimiser** – cherche le pattern : quelle structure de donnees ? quel invariant ? un tri prealable aide-t-il ?
5. **Coder** – ecris proprement en parlant a voix haute ; nomme clairement les variables.
6. **Tester** – deroule le code sur un exemple, verifie les cas limites, corrige.

Regle d'or

Annonce toujours la complexite (temps *et* espace) de ta solution, et propose une piste d'amelioration meme si tu n'as pas le temps de l'implementer.

2. Hash Map / Dictionnaire

Structure cle $\rightarrow$ valeur offrant un acces en $O(1)$ moyen. C'est l'outil le plus rentable en entretien : il transforme souvent un $O(n^2)$ en $O(n)$.

Quand l'utiliser

Compter des frequences ; retrouver un complement deja vu (Two Sum) ; grouper des elements par cle ; memoriser des indices deja rencontres.

Complexite : Temps $O(1)$ moyen | Espace $O(n)$

Problemes types : Two Sum (LC 1), Group Anagrams (LC 49), Top K Frequent (LC 347).

```
def two_sum(nums, target):
    seen = {} # valeur -> indice
    for i, x in enumerate(nums):
        if target - x in seen:
            return [seen[target - x], i]
        seen[x] = i
    return []
```

Piege : en cas de collision de cles, la derniere valeur ecrase la precedente. Reflexis a ce que la cle doit reellement représenter.

3. Hash Set

Ensemble d'elements uniques ; test d'appartenance en $O(1)$ moyen.

Quand l'utiliser

Tester l'appartenance rapidement ; detecter les doublons ; suivre les elements deja visites (parcours de graphe, cycles).

Complexite : Temps $O(1)$ moyen | Espace $O(n)$

Problemes types : Contains Duplicate (LC 217), Longest Consecutive (LC 128), Happy Number (LC 202).

```
def has_duplicate(nums):
    seen = set()
    for x in nums:
        if x in seen:
            return True
        seen.add(x)
    return False
```

Piege : un set ne conserve pas l'ordre d'insertion ; si l'ordre compte, utilise un dictionnaire (ordonne depuis Python 3.7) ou une liste annexe.

4. Two Pointers (deux pointeurs)

Deux indices qui avancent selon une regle. Deux variantes : *opposes* (left/right) ou *memme direction* (lent/rapide).

Quand l'utiliser

Tableau trie et recherche d'une paire ; palindrome ou inversion ; detection de cycle et recherche du milieu (lent/rapide).

Complexite : Temps $O(n)$ | Espace $O(1)$

Problemes types : Valid Palindrome (LC 125), Two Sum II (LC 167), 3Sum (LC 15).

```
def two_sum_sorted(nums, target):
    left, right = 0, len(nums) - 1
    while left < right:
        s = nums[left] + nums[right]
        if s == target:
            return [left, right]
        if s < target:
            left += 1
        else:
            right -= 1
    return []
```

Piege : la variante opposee suppose un tableau *trie*. Si l'enonce ne le garantit pas, il faut trier d'abord (+ $O(n \log n)$).

5. Sliding Window (fenetre glissante)

Maintenir une fenetre [`left`, `right`] et l'ajuster selon une contrainte, au lieu de recalculer chaque sous-tableau.

Quand l'utiliser

Sous-tableau ou sous-chaine *contigu* ; longueur min/max sous contrainte ; fenetre de taille fixe (somme de k elements consecutifs).

Complexite : Temps $O(n)$ | Espace $O(1)$ ou $O(k)$

Problemes types : Longest Substring (LC 3), Min Size Subarray (LC 209), Find Anagrams (LC 438).

```
def longest_unique(s):
    last = {} # caractere -> dernier indice vu
    left = best = 0
    for right, ch in enumerate(s):
        if ch in last and last[ch] >= left:
            left = last[ch] + 1
        last[ch] = right
        best = max(best, right - left + 1)
    return best
```

Piege : bien decider quand *agrandir* (avancer `right`) et quand *retrecir* (avancer `left`) la fenetre ; un ordre incorrect casse l'invariant.

6. Prefix Sum (somme prefixee)

On precalcule `prefix[i] = somme de nums[0..i-1]`. Toute somme d'intervalle se lit alors en $O(1)$: `sum(i, j) = prefix[j+1] - prefix[i]`.

Quand l'utiliser

Nombreuses requetes de somme sur intervalle ; compter les sous-tableaux de somme donnee (combine avec un hash map) ; indice pivot.

Complexite : Temps $O(n)$ precalcul, $O(1)$ requete | Espace $O(n)$

Problem types : Range Sum Query (LC 303), Subarray Sum = K (LC 560), Pivot Index (LC 724).

```
def subarray_sum_k(nums, k):
    count = running = 0
    seen = {0: 1}                # somme prefixee -> nb d'occurrences
    for x in nums:
        running += x
        count += seen.get(running - k, 0)
        seen[running] = seen.get(running, 0) + 1
    return count
```

Piege : initialise `seen = {0: 1}` pour compter les sous-tableaux qui commencent a l'indice 0.

7. Sorting (tri)

Trier revele souvent la structure cachee d'un probleme (voisinage, chevauchement). Cout de reference : $O(n \log n)$.

Quand l'utiliser

Regrouper ou detecter des doublons ; fusionner des intervalles ; preparer un two-pointers ; ordonner selon un critere personnalise.

Complexite : Temps $O(n \log n)$ | Espace $O(n)$ ou $O(1)$

Problem types : Merge Intervals (LC 56), Meeting Rooms (LC 252), Largest Number (LC 179).

```
def merge_intervals(intervals):
    intervals.sort(key=lambda p: p[0])
    res = []
    for start, end in intervals:
        if res and start <= res[-1][1]:
            res[-1][1] = max(res[-1][1], end)    # fusionner
        else:
            res.append([start, end])
    return res
```

Piege : le tri detruit l'ordre initial. Si tu dois rendre des indices d'origine, trie des paires (valeur, indice).

8. Binary Search (recherche dichotomique)

Recherche en $O(\log n)$ sur un espace trie ou monotone. Cle : trouver un *predicat monotone* (vrai puis faux, ou l'inverse).

Quand l'utiliser

Tableau trie ; reponse monotone (« la valeur x fonctionne-t-elle ? ») ; recherche de la premiere ou derniere position valide.

Complexite : Temps $O(\log n)$ | Espace $O(1)$

Problem types : Binary Search (LC 704), Search in Rotated Array (LC 33), Koko Bananas (LC 875).

```
def binary_search(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = left + (right - left) // 2    # evite le debordement
        if nums[mid] == target:
            return mid
        if nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

Piege : attention aux bornes ($<$ vs $<=$) et a la mise a jour de $\text{mid} \pm 1$ pour eviter les boucles infinies.

9. Stack (pile LIFO)

Last In, First Out. En Python : une simple `list` avec `append()` et `pop()`.

Quand l'utiliser

Parentheses / expressions equilibrees ; parsing et evaluation ; DFS iteratif ; fonctionnalites annuler / refaire.

Complexite : Temps $O(1)$ par operation | Espace $O(n)$

Problem types : Valid Parentheses (LC 20), Min Stack (LC 155), Evaluate RPN (LC 150).

```
def valid_parentheses(s):
    pairs = {"(": ")", "[": "]", "{": "}"
    stack = []
    for c in s:
        if c in pairs:
            if not stack or stack.pop() != pairs[c]:
                return False
        else:
            stack.append(c)
    return not stack
```

Piege : pense a verifier que la pile est *vide* a la fin ; une pile non vide signale des ouvrants non fermes.

10. Monotonic Stack (pile monotone)

Une pile dont les elements restent toujours croissants ou decroissants. Chaque element est empile et depile au plus une fois : cout total $O(n)$.

Quand l'utiliser

Prochain element plus grand / plus petit ; largest rectangle in histogram ; trapping rain water ; spans boursiers.

Complexite : Temps $O(n)$ | Espace $O(n)$

Problem types : Daily Temperatures (LC 739), Next Greater Element (LC 496), Largest Rectangle (LC 84).

```
def daily_temperatures(temps):
    res = [0] * len(temps)
    stack = [] # indices, temperatures décroissantes
    for i, t in enumerate(temps):
        while stack and temps[stack[-1]] < t:
            j = stack.pop()
            res[j] = i - j
        stack.append(i)
    return res
```

Piege : on empile souvent des *indices* plutôt que des valeurs, pour pouvoir calculer des distances.

11. Backtracking (retour sur trace)

Explorer toutes les possibilités via un schéma *choisir / recuser / annuler*. Coût souvent exponentiel.

Quand l'utiliser

Toutes les combinaisons, permutations ou sous-ensembles ; problèmes à contraintes (Sudoku, N-Queens) ; génération exhaustive.

Complexité : Temps $O(2^n)$ ou $O(n!)$ | Espace $O(n)$ pile de recursion

Problem types : Subsets (LC 78), Permutations (LC 46), Combination Sum (LC 39).

```
def subsets(nums):
    res, path = [], []
    def backtrack(start):
        res.append(path[:]) # copie de l'état courant
        for i in range(start, len(nums)):
            path.append(nums[i]) # choisir
            backtrack(i + 1)
            path.pop() # annuler (backtrack)
    backtrack(0)
    return res
```

Piege : enregistre une *copie* (`path[:]`), sinon toutes les solutions pointeront vers la même liste mutable.

12. Linked List (liste chaînée)

Des nœuds reliés par `.val` et `.next`. Les entretiens insistent sur la manipulation propre des pointeurs.

Quand l'utiliser

Inverser une liste ; détecter un cycle (lent/rapide) ; fusionner deux listes ; supprimer le *n*-ième nœud depuis la fin.

Complexité : Temps $O(n)$ | Espace $O(1)$

Problem types : Reverse List (LC 206), Linked List Cycle (LC 141), Merge Two Lists (LC 21).

```
def reverse_list(head):
    prev = None
    curr = head
    while curr:
        nxt = curr.next          # sauvegarder le suivant
        curr.next = prev        # inverser le lien
        prev = curr
        curr = nxt
    return prev
```

Piège : soigne les cas limites – liste vide, un seul noeud, mise à jour de **head**. Un *noeud sentinelle* (dummy) simplifie souvent le code.

13. Fiche recapitulative des complexites

A memoriser avant l'entretien :

Pattern	Temps	Espace
Hash Map / Set	$O(1)$ moyen	$O(n)$
Two Pointers	$O(n)$	$O(1)$
Sliding Window	$O(n)$	$O(k)$
Prefix Sum	$O(n)$ / $O(1)$	$O(n)$
Sorting	$O(n \log n)$	$O(n)$
Binary Search	$O(\log n)$	$O(1)$
Stack	$O(n)$	$O(n)$
Monotonic Stack	$O(n)$	$O(n)$
Backtracking	$O(2^n)$ / $O(n!)$	$O(n)$
Linked List	$O(n)$	$O(1)$

Checklist du jour J

- Parler a voix haute – le raisonnement est note.
- Clarifier les contraintes AVANT de coder.
- Annoncer la complexite temps + espace.
- Traiter les cas limites (vide, 1 element, negatifs, debordement).
- Nommer clairement ; code lisible > code astucieux.
- Bloque ? Revenir a la force brute, puis optimiser.
- Tester a la main avant de dire « c'est bon ».